

Custom Web Scraper

Deep Dive: Complete Technical Walkthrough

Explainer document generated for the project owner

Contents

1	What This Project Is	2
1.1	Key Terms, Defined Once	2
2	Architecture	2
2.1	Shared State (Module-Level Globals)	3
3	Directory Layout	3
4	Startup Sequence	4
5	Loading a Page	4
6	The Seven Locator Lookups	5
6.1	Dispatch Table	5
7	Formatting Results (Pure Layer)	6
8	The Results Window and Highlighting	7
9	Other UI Details	7
10	Dependencies	7
11	History: From Prior Versions to This One	8
12	Known Design Trade-offs (from the README)	8
13	Running It	9

1 What This Project Is

Custom Web Scraper is a desktop GUI application: you type a URL, it drives a real Chrome browser to that page, you pick one of seven ways to locate elements on the page (by CSS class, ID, name, tag, link text, partial link text, or XPath), and the tool shows every matching element in a results table (tag name, visible text, and key attributes). A “Highlight in Browser” button then outlines those exact elements in the live Chrome window so you can visually confirm the match.

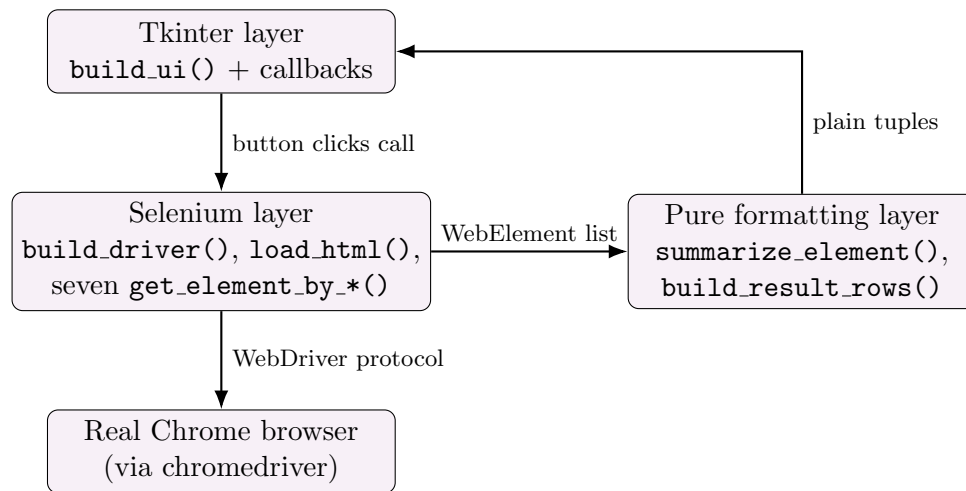
It exists so that small, one-off scraping questions (“what are all the `<a>` tags with this class on this page”) do not each require writing and running a throwaway Python script. Everything lives in a single file, `main.py`.

1.1 Key Terms, Defined Once

- **Web scraping.** Automatically extracting data from a web page, as opposed to a human reading it in a browser.
- **DOM (Document Object Model).** The tree-shaped, in-memory representation a browser builds of a web page’s HTML, which JavaScript on the page can add to or change after the initial page load.
- **Selenium.** A library (and protocol) for remote-controlling a real browser: opening it, navigating it, reading and clicking on elements, exactly as a human would, rather than parsing raw HTML text.
- **WebElement.** Selenium’s handle to one node in the live DOM (e.g. one `<a>` tag). Reading its text or attributes sends a command to the real browser and returns the current live value; the handle can go *stale* (become invalid) if the page changes underneath it.
- **Locator strategy.** The *method* used to find element(s) in the DOM: by exact CSS class, by ID attribute, by tag name, by exact/partial visible link text, by `name` attribute, or by XPath (a path-like query language for navigating an XML/HTML tree, e.g. `//div[@class="card"]/a`).
- **Tkinter / ttk.** Python’s built-in GUI toolkit. `ttk` is its “themed” widget set (buttons, comboboxes, tables) with a more modern look than classic Tkinter widgets.
- **chromedriver.** A small standalone program Selenium talks to, which in turn controls the actual Chrome browser process. Chrome and `chromedriver` versions must be compatible with each other.

2 Architecture

The file splits into three layers, sharing state through module-level variables rather than a class:



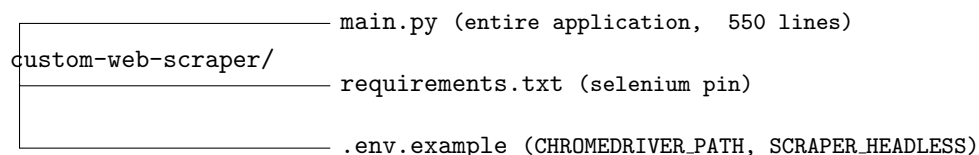
- **Tkinter layer.** `build_ui()` constructs every widget (URL panel, locator panel, status bar) and wires buttons to functions in the Selenium layer. It is a standalone function, not code directly under `if __name__ == "__main__":`, specifically so a test can call it and drive the resulting window without also starting a real Chrome browser.
- **Selenium layer.** Owns the single live `driver` object and every function that actually talks to the browser: loading a URL, and each of the seven locator lookups.
- **Pure formatting layer.** `summarize_element()` and `build_result_rows()` take `WebElement`-like objects and return plain Python tuples, no network or driver access, so this layer alone can be unit tested with fake stand-in objects.

2.1 Shared State (Module-Level Globals)

Variable	Holds	Read/written by
<code>driver</code>	the live Selenium/Chrome session	every Selenium-layer function
<code>is_request_loaded</code>	whether a page has been loaded yet	<code>load_html</code> , every locator lookup
<code>last_elements</code>	most recent match list	<code>show_data</code> , <code>highlight_in_browser</code>
Tkinter widgets (<code>root</code> , <code>entries</code> , etc.)	the actual GUI objects	callback functions directly

This is a deliberate simplicity trade-off for a single-window script: there is exactly one browser session and one window, so a class hierarchy to pass state around would add structure without solving a problem this program actually has. Section 7 (“What I’d Do Differently”, carried over from the README) notes the real cost: none of the seven locator functions can be unit-tested without a live driver and window, because they all read the same globals directly.

3 Directory Layout



No templates, no static assets, no database: the entire UI is built programmatically with Tkinter widgets in Python, not markup.

4 Startup Sequence

```
if __name__ == "__main__":
    driver = build_driver()
    build_ui()
    root.mainloop()
    driver.quit()
```

1. `build_driver()` configures Chrome options (headless or not, based on the `SCRAPER_HEADLESS` environment variable), builds a Selenium **Service** (optionally pointed at an explicit `chromedriver` binary via `CHROMEDRIVER_PATH`), starts Chrome through it, maximizes the window, and returns the live `driver` handle. If `CHROMEDRIVER_PATH` is unset, **Selenium Manager** (bundled since Selenium 4.6) automatically downloads and caches a `chromedriver` build matching the installed Chrome version, which is why the app needs a one-time network connection the very first time it runs on a machine.
2. `build_ui()` constructs the whole Tkinter window and returns it as `root` (Section 6).
3. `root.mainloop()` is Tkinter's blocking event loop: it waits for and dispatches user input (clicks, keystrokes) until the window is closed. Nothing after this line runs until the window closes.
4. `driver.quit()` closes the Chrome session once the GUI window is closed, so no orphaned browser process is left running.

5 Loading a Page

```
def load_html():
    global is_request_loaded
    url = request_url_entry.get().strip()
    set_status("Loading page...", "info")
    root.update_idletasks()
    try:
        driver.get(url)
        is_request_loaded = True
    except InvalidArgumentException:
        set_status("Please enter a valid URL", "error")
        return
    try:
        WebDriverWait(driver, 10).until(ec.presence_of_element_located((By.CSS_SELECTOR, "html")))
        WebDriverWait(driver, 10).until(ec.presence_of_element_located((By.CSS_SELECTOR, "body")))
    except TimeoutException:
        set_status("Network error: the page did not load in time", "error")
        return
    add_to_history(request_url_entry, url)
    set_status(f"Loaded {url}", "success")
```

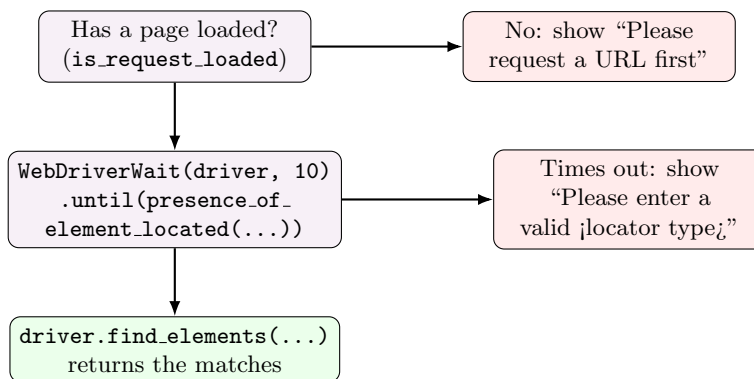
`root.update_idletasks()` forces Tkinter to redraw the "Loading page..." status text immediately, rather than waiting until the whole function returns (GUI frameworks normally only repaint between

event-loop iterations, so a long-running call inside one callback would otherwise freeze the visible text).

`driver.get(url)` sends the navigation command to Chrome. This call returns once the browser's initial `load` event fires, but that does not guarantee JavaScript-injected content has finished rendering, which is why the code then explicitly waits (up to 10 seconds each) for `<html>` and `<body>` to be present using `WebDriverWait` plus an **expected condition** (`presence_of_element_located`): a small, reusable predicate Selenium polls repeatedly until it is true or a timeout is hit, which is more reliable against variable page load times than a fixed `time.sleep()`.

An invalid URL (e.g. malformed syntax) raises Selenium's `InvalidArgumentException` before any waiting even starts; a URL that is syntactically valid but never finishes loading (dead host, slow network) instead falls through to the `TimeoutException` branch after 10 seconds. Either way the user gets a specific, human-readable status message instead of a stack trace or a frozen window.

6 The Seven Locator Lookups



The seven functions — `get_element_by_class_name`, `get_element_by_id`, `get_element_by_partial_link_text`, `get_element_by_link_text`, `get_element_by_tag_name`, `get_element_by_xpath`, and `get_element_by_name` — are structurally identical, each differing only in which `By.*` constant it passes to `WebDriverWait/find_elements` and which locator type it names in its error message. Each:

1. Bails out immediately with "Please request a URL first" if `is_request_loaded` is still `False`.
2. Waits up to 10 seconds for at least one matching element to be **present** in the DOM (not necessarily visible, just present).
3. On success, calls `driver.find_elements(...)` (plural: returns a list, possibly of more than one match, unlike `find_element` singular which raises if there is not exactly one).
4. On a 10-second timeout, reports a locator-specific "please enter a valid ..." message and returns `(None, None)`.
5. Returns `(True, list_of_elements)` on success.

6.1 Dispatch Table

```
LOCATOR_DISPATCH = {
    "Class": get_element_by_class_name,
    "ID": get_element_by_id,
    "Partial Link Text": get_element_by_partial_link_text,
    "Link Text": get_element_by_link_text,
    "Name": get_element_by_name,
```

```

    "Tag Name": get_element_by_tag_name,
    "XPath": get_element_by_xpath,
}

```

`on_get_elements_click()` reads the selected locator strategy from a Tkinter dropdown, looks the matching function up in this dictionary, and calls it with the typed-in locator value, rather than a long chain of `if/elif` statements comparing the dropdown value to each strategy name. This is a common Python pattern: a dict of callables replaces a branch-per-case chain when every branch just calls a different function with the same arguments.

7 Formatting Results (Pure Layer)

```

def _safe_call(fn):
    try:
        return fn()
    except Exception:
        return None

def summarize_element(element, index, text_limit=80):
    tag = _safe_call(lambda: element.tag_name) or "?"
    text = _safe_call(lambda: element.text) or ""
    text = " ".join(text.split())
    if len(text) > text_limit:
        text = text[: text_limit - 1] + "..."
    attr_parts = []
    for attr in ATTRIBUTES_TO_SHOW:
        value = _safe_call(lambda a=attr: element.get_attribute(a))
        if value:
            attr_parts.append(f"{attr}={value}")
    attributes = ", ".join(attr_parts)
    return index, tag, text, attributes

```

- **Why `_safe_call` exists.** A `WebElement` handle can go **stale** (raise an exception on *any* attribute access) if the live page changes between when the element was found and when it is read, e.g. a script on the page re-renders part of the DOM. `_safe_call` wraps each individual field read (tag name, text, each attribute) so one stale or missing field falls back to `None`/empty instead of throwing away the whole row.
- **Whitespace collapsing.** `" ".join(text.split())` is a common Python idiom: `str.split()` with no arguments splits on any run of whitespace (including newlines and tabs) and discards empty pieces, so joining the pieces back with single spaces collapses “Hello\n world” into “Hello world”.
- **Text truncation.** Text longer than `text_limit` (default 80 characters) is cut to `text_limit - 1` characters plus an ellipsis character (...), so the results table stays readable regardless of how much text an element contains.
- **Attribute allowlist.** Only a fixed set of commonly useful attributes (`id`, `class`, `name`, `href`, `src`, `type`, `value`) is checked per element, rather than every attribute the element happens to have, keeping the summary short and consistent.
- **Why this layer is separately testable.** Neither function touches `driver` or any Tkinter object; they only call read-only methods (`.tag_name`, `.text`, `.get_attribute()`) on whatever object is

passed in. Because Python does not enforce types at the language level (a concept sometimes called **duck typing**: “if it walks like a duck and quacks like a duck”), a plain fake object exposing those same three members is indistinguishable from a real `WebElement` to this code, which is what makes it unit-testable without a real browser.

8 The Results Window and Highlighting

`show_data()` opens a Tkinter `Toplevel` (a secondary window, separate from the main app window) containing a `ttk.Treeview`: a built-in scrollable, column-based table widget. Each formatted row from `build_result_rows()` becomes one `Treeview` row. The raw `WebElement` list is also stashed in the `last_elements` global so the “Highlight in Browser” button can act on the same elements without re-querying the page (a second query could, in principle, return different or reordered results if the page had changed in the meantime).

`highlight_in_browser()` loops over `last_elements` and, for each one, runs a small JavaScript snippet via `driver.execute_script(...)` that sets a CSS outline and scrolls the element into view. Running arbitrary JavaScript inside the already-loaded page like this is a standard Selenium capability, used here purely for visual confirmation, not for altering page behavior. Each call is individually wrapped in a `try/except` so one stale element (if the page changed since the lookup) does not stop the rest from being highlighted.

9 Other UI Details

- **Dropdown-driven instructions.** `dropdown_var_detect()` is attached to the locator-strategy dropdown’s `StringVar` via `.trace("w", ...)`, a Tkinter mechanism that calls a function every time a variable’s value changes. It updates a label with a hint describing the expected input format (e.g. “Enter the class name...”) for whichever locator strategy is currently selected.
- **History comboboxes.** `add_to_history()` pushes a value to the front of a `ttk.Combobox`’s drop-down list, removing a duplicate if present and capping the list at 10 entries, applied to both the URL field and the locator-value field after every successful lookup. This history lives only in memory and resets when the app restarts (a known limitation stated plainly in the README rather than glossed over).
- **Status bar.** `set_status(message, kind)` is the single choke point for user feedback: every code path (loading, found N elements, every error) routes through it, coloring the text by severity (info/success/error) via a small `STATUS_COLORS` dictionary, so the user is never left staring at a blank or frozen-looking window.
- **Styling.** A dark purple theme is applied through `ttk.Style` with named styles (`Panel.TFrame`, `Heading.TLabel`, etc.) rather than repeating color constants at every widget call site.

10 Dependencies

Package	Pinned range	Role
selenium	>=4.15,<5	Browser automation: driving Chrome, locating elements, waits
tkinter/ttk	(Python standard library)	GUI toolkit; not in <code>requirements.txt</code>

Selenium 4.6+ specifically matters because that is the version range that bundles **Selenium Manager**, removing the need to manually download and version-match a **chromedriver** binary (Section 9 explains why this project’s earlier, hardcoded-path version was less portable).

11 History: From Prior Versions to This One

The README documents several concrete changes made versus an earlier version of this tool (referred there as `temp.py/temp1.py` in source history):

- **Backend swap: requests + BeautifulSoup → Selenium.** The original approach fetched raw HTML with an HTTP client and parsed it with BeautifulSoup using CSS selectors. That approach cannot see content injected by JavaScript after the initial page load (increasingly common on modern sites), and a nonexistent CSS selector only ever raised an easy-to-catch parse error. Moving to Selenium fixed the JavaScript-rendering gap, but introduced a new failure mode: a nonexistent Selenium locator does not raise a parse error at all, it just waits and then times out, which is why every lookup here is wrapped in `WebDriverWait(...)` with an explicit `TimeoutException` handler.
- **Hardcoded Windows path removed.** The original `main.py` pointed directly at `C:\Development\chromedriver` which only worked on the original author’s machine. This version resolves the driver automatically via Selenium Manager, with `CHROMEDRIVER_PATH` as an optional explicit override, so the same code runs unmodified on Linux, macOS, or Windows.
- **The “page not loaded yet” guard.** Every locator function checks `is_request_loaded` first. Without it, clicking “Get Elements” before “Request” would call `driver.find_elements` on an uninitialized or blank page state and surface a raw exception instead of a helpful message.

What was actually verified, stated plainly. This is a GUI application that drives a real Chrome browser, so a full end-to-end run needs both a display and outbound network access (the first run needs to download a matching **chromedriver** via Selenium Manager, unless `CHROMEDRIVER_PATH` points at one already present). The sandbox this project was verified in has *no outbound network access*, so an actual `driver.get(url)` call against a live website, and the “Highlight in Browser” JavaScript against a real page, were never exercised here. What *was* verified in that sandbox: `main.py` compiles cleanly; the pure formatting functions (`summarize_element`, `build_result_rows`) pass a unit test suite using fake duck-typed objects (normal elements, truncation, whitespace collapsing, missing attributes, and a simulated stale-element exception); and the full Tkinter window, including history comboboxes, the status bar, and the highlight button’s enabled/disabled state, was constructed and driven end-to-end against a *fake* driver object on a live X display, confirming the UI wiring itself (not the real browser calls) behaves as designed. The Selenium API surface used (`webdriver.Chrome`, `WebDriverWait`, `By.*`, `find_elements`, `execute_script`) is standard, stable Selenium 4.x API matching its documented usage, but that is a statement about the API being well-established, not a substitute for having actually run it against a live site here.

12 Known Design Trade-offs (from the README)

- **Shared globals over a class.** All seven locator functions read the same module-level `driver` and `is_request_loaded` rather than receiving them as arguments. Simplest way to wire seven near-identical lookups to one dispatch table, but none of them can be unit-tested without the whole window and driver, and the design could not support scraping two pages at once.

- **In-memory history only.** URL and locator history reset on restart; no persistence to disk.
- **No results export.** The table is view-only; no CSV export or copy-to-clipboard.
- **Duplicated per-locator boilerplate.** The seven `get_element_by_*` functions are reached through a dispatch dict instead of an `if/elif` chain, but each still repeats the same `is_request_loaded/WebDriverWait/T` shape; a fix to that shape has to be copied into all seven by hand.

13 Running It

```
python3 -m venv venv
source venv/bin/activate # venv\Scripts\activate on Windows
pip install -r requirements.txt
python main.py
```

Requires Python 3.9+, a Chrome/Chromium install, and (the first time) outbound network access so Selenium Manager can fetch a matching `chromedriver`. On Debian/Ubuntu, `tkinter` may need a separate system package: `sudo apt install python3-tk`.